Cloud Computing Assignment

Name: Abhinav Kumar Jha
Roll: 23010144
Github Repo:- github.com/Abhinav458/Cloud-Computing-Assignment

- Header, Link, Core Learning & Architecture.
  - Git Tag: final
  - Release Tag: v1.0.0
  - Cloud Native ML Serving: Learned how to decouple container images from heavy ML model artifacts (model-v1.tar.gz) by pulling weights dynamically at container cold-start from an in-cluster S3 store(seaweedFS in namespace data) rather than baking multi-hundred-MB weights in to Docker layer.
  - Kubernetes Resource Governance: Understood how Linux cgroups enforce Kubernetes requests (250m CPU, 512Mi RAM) for scheduling & hard limits (100m CPU, 1Gi RAM) where memory overruns trigger OOMKill (Exit Code 137) & CPU bursts trigger CFS throttling.
  - MLOps Quality Gating: Learned why standard unit tests (SKIP_MODEL_LOAD=1 pytest) are insufficient for ML microservices & how automated validation gates (--min-validity 95) block broken or untrained models(Pythia-70M) in CI/CD.
- VLSM & Stateless Firewalls: Mastered partitioning 10.20.0.0/16 into /22, /24, /25, and /27 CIDR blocks and configuring stateless NACLs requiring all the explicit ephmeral return rules (1024-65535).

- Technical Challenges faced & Solutions Implemented
  - Challenge 1: Namespace Mismatch & Empty Telemetry Logs.
    - → Problem: Initial kubectl rollout & pod recovery commands targeted the default/app, namespace, resulting in 0-byte log files in results/.
    - → Solution: Audited k8s/deployment.yaml, identified that common-parser runs in the pipeline namespace (-n pipeline) while SeaweedFS runs in -n data, & re-executed rollout, undo and pod-detection self-healing captures.
  - Challenge 2: Stateless NACL schema & Ephemeral Port Handling.
    - → Problem: tools/nacl-sim.py rejected network/nacl.json due to schema mismatches (rule_no vs rule and port_range array vs port_form/port_to) & JSON array vs JSONL packet inputs.
    - → Solution: Refactored nacl.json to use integer rule priorities (50 for explicit blacklist 198.51.100.0/24 before 100 for HTTP/HTTPS 0.0.0.0/0) and explicit port_from/port_to keys, verifying all rules against nacl_sim.py.
  - Challenge 3: Cold-start Latency vs Readiness Probes
    - → Problem: Downloading and extracting model weights from SeaweedFS during the pod initialization introduces cold-start delay; premature traffic routing causes the several HTTP errors likes 502/503 during rolling updates.
    - → Solution: Configured Kubernetes /healthz readiness probes alongside the RollingUpdate (maxSurge: 25%, maxUnavailable: 0) so the old V1 pods continue serving all 100% of the requests until the V2 pod finishes the model extraction & ultimately increased the further process.

- Key Empirical Findings (Measured Results):

1) Model Accuracy & Validity: fine-tuned student-v1 acheived 100% JSON schema Validity on data/validation-jsonl, passing the ≥ 95% quality gate, whereas untrained pythia-70M failed the gate as expected.

2) Memory Headroom Under Load: Under strict 1vCPU/1 GiB limits across 50 sequential requests, peak container memory reached 226.9 MiB (22.16% of the 1 GiB limit), leaving 797.1 MiB (77.84%) safety margin with 0 OOM kills.

3) High-Concurrency Resilience: Locust load testing completed 126/126 # requests (0% failure), and v1→v2 rolling updates + rollbacks achieved zero dropped /healthz checks.

- Limitations & Proposed Improvements:

1) Model Caching via PVC: Currently, every new pod downloads model-v1.tar.gz from SeaweedFS on cold start. Mouting a ReadOnlyMany PersistentVolumeClaim (PVC) or using an InitContainer with shared node cache would cut cold-start latency by >80%.

2) Quantization & Batching: Applying INT8/ONNX Runtime quantization and dynamic request batching would increase CPU inference throughput under the vCPU quota.

3) Network Policy Enforcement: Supplementing the VPC-level stateless NACLs with the Kubernetes NetworkPolicy (via Cilium/Calico CNI) to restrict pod-to-pod traffic between pipeline & data namespaces.